

LECTURE 10

데이터 분석

경북대학교
전자공학부
김현수

목차

10.1 판다스 데이터프레임

10.2 데이터 전처리

10.3 판다스 함수

10.4 파일 데이터 이용

10.5 [플러스 예제] 자동차 배기량과 CO2 배출량의 상관관계

학습목표

- 판다스 데이터프레임의 개념을 설명할 수 있습니다.
- 데이터프레임을 생성하고 특정 데이터에 접근하거나 데이터를 조작할 수 있습니다.
- 데이터프레임을 시각화할 수 있습니다.
- 판다스를 이용해서 파일을 불러오거나 저장할 수 있습니다.

판다스 데이터프레임

1. 판다스 데이터프레임

■ 데이터프레임의 개요

- 판다스(Pandas): 데이터를 처리하거나 분석할 때 사용하는 라이브러리
- 판다스는 넘파이에서 파생되어 만들어졌기 때문에 넘파이 배열 구조를 기반으로 함
- 판다스를 사용하려면 라이브러리를 불러와야 하는데 이때 일반적으로 pd라는 별칭을 사용

```
import pandas as pd
```

1. 판다스 데이터프레임

■ 데이터프레임의 개요

- 판다스는 1차원 배열 구조인 '시리즈'와 2차원 배열 구조인 '데이터프레임'이라는 데이터 구조를 제공
- 시리즈(series)는 인덱스와 값으로 구성된 1차원 배열 구조이며 딕셔너리와 유사
- 데이터프레임(dataframe)은 2차원 배열 구조로 테이블 형식의 데이터를 저장하고 처리하는 데 사용됨
- 데이터프레임은 행과 열을 가진 2차원 배열로, 열(column)은 '속성'이라고도 함

1. 판다스 데이터프레임

■ 데이터프레임의 개요

- 예를 들어, 학생 성적을 관리하는 데이터프레임에는 이름, 시험 점수, 과제1 점수, 과제2 점수와 같은 열이 있을 수 있고, 각 행은 개별 학생의 정보를 나타냄

		열				
인덱스	이름	행 인덱스	이름	시험 점수	과제1 점수	과제2 점수
0	Jessica	0	Jessica	45	20	19
1	Liam	1	Liam	30	17	14
2	Sophia	2	Sophia	40	22	18
3	Ryan	3	Ryan	37	18	18
4	Alex	4	Alex	48	24	25

값

(a) 시리즈

행

(b) 데이터프레임

그림 11-1 판다스 시리즈와 데이터프레임의 구조

1. 판다스 데이터프레임

■ 데이터프레임 생성하기

- 데이터프레임을 생성할 때는 DataFrame 함수를 사용

| Syntax | DataFrame() 함수

```
import pandas as pd  
pd.DataFrame(데이터)
```

데이터프레임을 생성한다.

1. 판다스 데이터프레임

■ 데이터프레임 생성하기

- [코드 11-1]은 데이터프레임을 생성하는 코드
- DataFrame 함수 안에 들어갈 데이터는 일반적으로 딕셔너리를 사용
- 키에 해당하는 name, test, assign1, assign2는 열 이름이 되고, 값은 열에 대한 각 행의 데이터로 리스트 형식을 사용

코드 11-1

```
01 import pandas as pd
02
03 df = pd.DataFrame({'name': ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex'],
04                   'test': [45, 30, 40, 37, 48],
05                   'assign1': [20, 17, 22, 18, 24],
06                   'assign2': [19, 14, 18, 15, 25]})
```

열 이름

열 안의 각 행 데이터

```
df = pd.DataFrame( {'name': ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex'],
                    'test': [45, 30, 40, 37, 48],
                    'assign1': [20, 17, 22, 18, 24],
                    'assign2': [19, 14, 18, 15, 25] } )
```

그림 11-2 데이터프레임을 생성

1. 판다스 데이터프레임

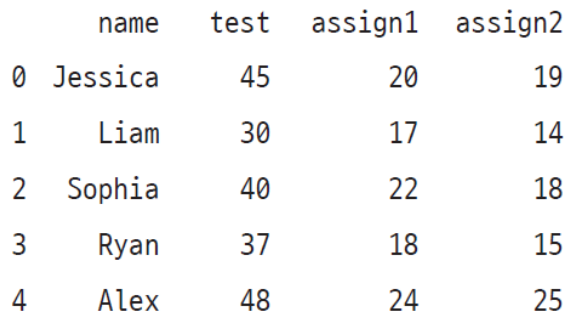
■ 데이터프레임 생성하기

■ 데이터프레임 출력하기

- 생성한 df 데이터프레임을 print 함수로 출력하면 다음과 같음

코드 11-2

```
01 print(df)
```



	name	test	assign1	assign2
0	Jessica	45	20	19
1	Liam	30	17	14
2	Sophia	40	22	18
3	Ryan	37	18	15
4	Alex	48	24	25

1. 판다스 데이터프레임

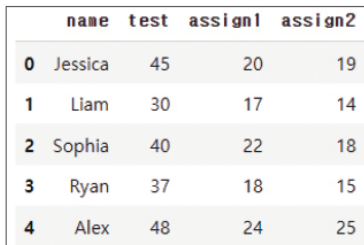
■ 데이터프레임 생성하기

■ 데이터프레임 출력하기

- display 함수를 이용해서 데이터프레임을 출력할 수도 있음
- 실행 결과에서 확인할 수 있듯이 보기 좋게 표 형식으로 출력됨
- 데이터프레임 각 행 왼쪽의 번호를 행 인덱스(index)라고 함

코드 11-3

```
01 display(df)
```



	name	test	assign1	assign2
0	Jessica	45	20	19
1	Liam	30	17	14
2	Sophia	40	22	18
3	Ryan	37	18	15
4	Alex	48	24	25

1. 판다스 데이터프레임

■ 데이터프레임 생성하기

■ 리스트와 열 이름을 설정하여 데이터프레임 생성하기

- 다음은 다른 방법으로 데이터프레임을 생성하는 코드
- 데이터는 각 학생의 개별 정보로 구성된 데이터들의 리스트로 설정하고 열 이름은 columns 매개변수를 이용해서 설정

코드 11-4

```
01 import pandas as pd
02
03 df = pd.DataFrame([['Jessica',45,20,19],['Liam',30,17,14],['Sophia',40,22,18],
04                   ['Ryan',37,18,15],['Alex',48,24,25]],
05                   columns=['name','test','assign1','assign2'])
```

```
df = pd.DataFrame([['Jessica',45,20,19],['Liam',30,17,14],['Sophia',40,22,18],
    데이터 ..... ['Ryan',37,18,15],['Alex',48,24,25]],
    열 이름 설정 ..... columns=['name','test','assign1','assign2'] )
```

그림 11-3 리스트와 columns 매개변수로 데이터프레임을 생성

1. 판다스 데이터프레임

■ 데이터프레임 생성하기

■ 빈 데이터프레임을 생성하고 열 추가하기

- 빈 데이터프레임을 생성한 후에 열을 추가해서 데이터프레임을 생성해도 됨

| Syntax | 데이터프레임에 열 추가하기

```
데이터프레임[열 이름] = 데이터
```

데이터프레임에 '열 이름' 열을 추가합니다. 해당 열의 데이터는 '데이터'가 됩니다.

1. 판다스 데이터프레임

■ 데이터프레임 생성하기

■ 빈 데이터프레임을 생성하고 열 추가하기

- 데이터프레임에 열을 추가하는 방법

코드 11-5

```
01 import pandas as pd
02
03 df = pd.DataFrame()
04 df['name'] = ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex']
05 df['test'] = [45, 30, 40, 37, 48]
06 df['assign1'] = [20, 17, 22, 18, 24]
07 df['assign2'] = [19, 14, 18, 15, 25]
```

04행

	name
0	Jessica
1	Liam
2	Sophia
3	Ryan
4	Alex

05행

	name	test
0	Jessica	45
1	Liam	30
2	Sophia	40
3	Ryan	37
4	Alex	48

그림 11-4 데이터프레임에 열을 추가하는 과정

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 열 추출하기

- 데이터프레임에서 특정 열을 추출하려면 대괄호([]) 안에 열 이름을 넣으면 됨
- df 데이터프레임을 생성하고 'name' 열을 추출하여 출력하는 코드는 [코드 11-6]과 같음

코드 11-6

```
01 import pandas as pd
02
03 df = pd.DataFrame({'name': ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex'],
04                   'test': [45, 30, 40, 37, 48],
05                   'assign1': [20, 17, 22, 18, 24],
06                   'assign2': [19, 14, 18, 15, 25]})
07
08 print(df['name'])
```

```
0  Jessica
1    Liam
2   Sophia
3     Ryan
4     Alex
```

Name: name, dtype: object

1. 판다스 데이터프레임

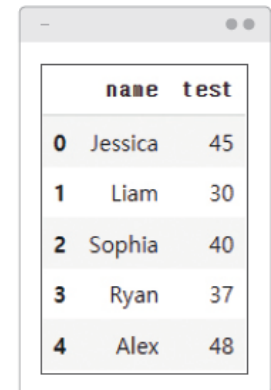
■ 데이터에 접근하기

▪ 여러 개의 열 추출하기

- 여러 개의 열을 추출하려면 대괄호([]) 안에 열 이름들로 이루어진 리스트를 넣으면 됨
- [코드 11-7]은 'name'과 'test' 두 개의 열을 추출하여 출력

코드 11-7

```
01 display(df[['name','test']])
```



	name	test
0	Jessica	45
1	Liam	30
2	Sophia	40
3	Ryan	37
4	Alex	48

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 여러 개의 열 추출하기

- pop 함수를 사용해서 열을 추출할 수도 있는데 이때 추출한 열은 데이터프레임에서 삭제됨

| Syntax | pop() 함수

```
데이터프레임.pop(열 이름)
```

‘데이터프레임’에서 ‘열 이름’ 열을 추출하여 반환하고 ‘열 이름’ 열은 삭제한다.

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 여러 개의 열 추출하기

- [코드 11-8]은 df 데이터프레임에서 'name' 열을 추출하여 n에 저장하는데, df 데이터프레임에서 'name' 열이 삭제됨

코드 11-8

```
01 n = df.pop('name')
```

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 데이터프레임 순회하기

- 데이터프레임의 모든 행을 순회할 때 `iterrows` 함수를 사용

| Syntax | `iterrows()` 함수

```
for 인덱스, 행 in 데이터프레임.iterrows():  
    문장
```

데이터프레임 행의 수만큼 반복한다. 반복할 때마다 '인덱스'에는 인덱스가 저장되고, '행'에는 각 행의 데이터가 순서대로 저장된다.

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 데이터프레임 순회하기

- [코드 11-9]는 iterrows 함수를 이용해서 df 데이터프레임의 인덱스, name 값, test 값을 한 개씩 출력하는 예

코드 11-9

```
01 for index, row in df.iterrows():  
02     print(index, row['name'], row['test'])
```

```
0 Jessica 45  
1 Liam 30  
2 Sophia 40  
3 Ryan 37  
4 Alex 48
```

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 데이터프레임 순회하기

- [코드 11-9]는 iterrows 함수를 이용해서 df 데이터프레임의 인덱스, name 값, test 값을 한 개씩 출력하는 예

0	row	'Jessica'	45	20	19
index		row['name']	row['test']	row['assign1']	row['assign2']

그림 11-5 첫 번째 행의 인덱스와 데이터

1	row	'Liam'	30	17	14
index		row['name']	row['test']	row['assign1']	row['assign2']

그림 11-6 두 번째 행의 인덱스와 데이터

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 데이터프레임 연산하기

- 데이터프레임도 넘파이 배열처럼 연산을 수행할 수 있음
- 데이터프레임 간의 연산은 물론 데이터프레임에 포함된 열 데이터 간의 연산도 가능
- 다음은 df 데이터프레임에서 각 학생들에 대한 성적인 'test', 'assign1', 'assign2' 열의 합을 구해 'sum' 열에 저장하는 코드

코드 11-10

```
01 df['sum'] = df['test']+df['assign1']+df['assign2']  
02 display(df)
```

	name	test	assign1	assign2	sum
0	Jessica	45	20	19	84
1	Liam	30	17	14	61
2	Sophia	40	22	18	80
3	Ryan	37	18	15	70
4	Alex	48	24	25	97

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 비교 연산하여 조건을 만족하는 데이터 추출하기

- 비교 연산을 이용하면 데이터프레임에서 특정 조건을 만족하는 데이터만 추출할 수 있음
- 다음은 'sum' 열의 값이 80 이상인 조건을 만족하는 데이터 행을 추출하는 코드

코드 11-11

```
01 display(df[df['sum']>=80])
```

	name	test	assign1	assign2	sum
0	Jessica	45	20	19	84
2	Sophia	40	22	18	80
4	Alex	48	24	25	97

1. 판다스 데이터프레임

■ 데이터에 접근하기

■ 비교 연산하여 조건을 만족하는 데이터 추출하기

- 그리고 [코드 11-12]는 'sum' 열의 값이 80 이상인 조건을 만족하는 데이터 행의 'name' 열과 'sum' 열만 추출

```
df[df['sum']>=80][['name','sum']]
```

조건

추출할 열

그림 11-7 조건을 만족하는 데이터의 특정 열을 추출

코드 11-12

```
01 display(df[df['sum']>=80][['name','sum']])
```

	name	sum
0	Jessica	84
2	Sophia	80
4	Alex	97

데이터 전처리

2. 데이터 전처리

■ 데이터 조작하기

▪ 열 이름 변경하기

- rename 함수를 이용하면 열 이름을 변경할 수 있음

| Syntax | rename() 함수

```
데이터프레임.rename(columns={'열 이름':'새로운 열 이름'})
```

‘열 이름’을 ‘새로운 열 이름’으로 변경한다. 여기에 inplace=True 옵션을 추가하면 데이터프레임에 변경된 열 이름이 바로 반영된다.

2. 데이터 전처리

■ 데이터 조작하기

■ 열 이름 변경하기

- [코드 11-13]은 df 데이터프레임의 'test' 열 이름을 'exam'으로 변경

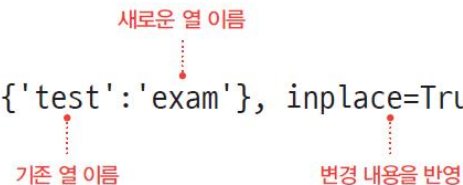
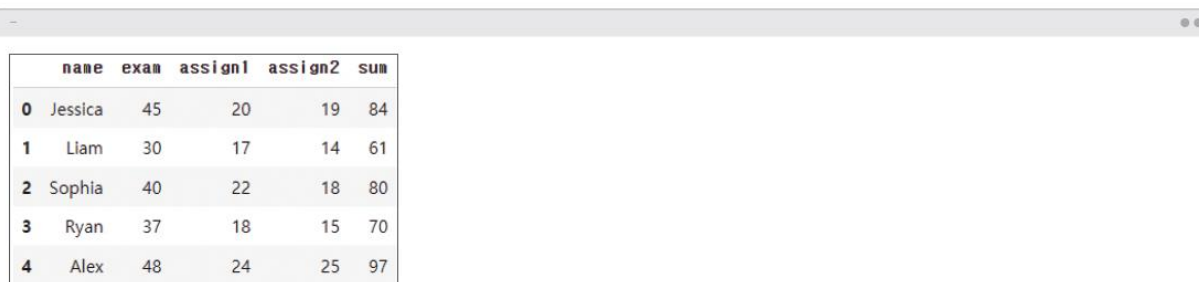

`df.rename(columns={'test':'exam'}, inplace=True)`

그림 11-8 데이터프레임의 열 이름을 변경

코드 11-13

```
01 df.rename(columns={'test':'exam'}, inplace=True)
02 display(df)
```



	name	exam	assign1	assign2	sum
0	Jessica	45	20	19	84
1	Liam	30	17	14	61
2	Sophia	40	22	18	80
3	Ryan	37	18	15	70
4	Alex	48	24	25	97

2. 데이터 전처리

■ 데이터 조작하기

■ 데이터 정렬하기

- `sort_values` 함수를 이용하면 데이터를 정렬할 수 있음

| Syntax | `sort_values()` 함수

```
데이터프레임.sort_values('열 이름')
```

‘열 이름’을 기준으로 데이터를 정렬한다. 기본적으로 오름차순으로 정렬하고, `ascending=False` 옵션을 추가하면 내림차순으로 정렬한다.

2. 데이터 전처리

■ 데이터 조작하기

■ 데이터 정렬하기

- [코드 11-14]는 df 데이터프레임을 'sum' 열을 기준으로 내림차순으로 정렬하여 sort_df에 저장함

코드 11-14

```
01 sort_df = df.sort_values('sum', ascending=False)
02 display(sort_df)
```

	name	exam	assign1	assign2	sum
4	Alex	48	24	25	97
0	Jessica	45	20	19	84
2	Sophia	40	22	18	80
3	Ryan	37	18	15	70
1	Liam	30	17	14	61

```
df.sort_values('sum', ascending=False)
```

내림차순 옵션

정렬 기준이 되는 열

그림 11-9 특정 열을 기준으로 데이터프레임을 정렬

2. 데이터 전처리

■ 데이터 조작하기

■ 조건에 따라 다른 값 부여하기

- 데이터 조건에 따라 값이 다른 열을 생성하려면 넘파이의 `where` 함수를 이용
- [코드 11-15]는 'grade' 열을 새롭게 생성하고 'sum' 열의 값이 90 이상이면 'A'를, 그렇지 않고 80 이상이면 'B'를, 그렇지 않으면 'C'를 'grade' 열의 값으로 부여함

코드 11-15

```
01 import numpy as np
02
03 df['grade'] = np.where(df['sum']>=90, 'A',
04                       np.where(df['sum']>=80, 'B', 'C'))
05 display(df)
```

	name	exam	assign1	assign2	sum	grade
0	Jessica	45	20	19	84	B
1	Liam	30	17	14	61	C
2	Sophia	40	22	18	80	B
3	Ryan	37	18	15	70	C
4	Alex	48	24	25	97	A

2. 데이터 전처리

■ 데이터 조작하기

■ 조건에 따라 다른 값 부여하기

`df['grade'] = np.where(df['sum']>=90, 'A',
np.where(df['sum']>=80, 'B', 'C'))`

조건식1 조건식1이 참이면 선택될 값
조건식1이 거짓이고 조건식2가 참이면 선택될 값
조건식2 모든 조건식이 거짓이면 선택될 값

그림 11-10 데이터 조건에 따라 다른 값 부여

2. 데이터 전처리

■ 데이터 조작하기

▪ 조건에 따라 다른 값 부여하기

- [코드 11-16]은 'sum' 열의 값이 80 이상이면 'pass'를, 그렇지 않으면 'fail'을 값으로 갖는 'result' 열을 추가함

코드 11-16

```
01 df['result'] = np.where(df['sum']>=80, 'pass', 'fail')
02 display(df)
```

	name	exam	assign1	assign2	sum	grade	result
0	Jessica	45	20	19	84	B	pass
1	Liam	30	17	14	61	C	fail
2	Sophia	40	22	18	80	B	pass
3	Ryan	37	18	15	70	C	fail
4	Alex	48	24	25	97	A	pass

df['result'] = np.where(df['sum']>=80, 'pass', 'fail')

조건식

조건식이 참이면 선택될 값

조건식이 거짓이면 선택될 값

그림 11-11 데이터프레임 조건에 따라 값이 다른 열 추가

2. 데이터 전처리

■ 데이터 조작하기

▪ 값의 빈도수 구하기

- value_counts 함수를 이용하면 특정 열에서 값의 빈도수를 셀 수 있음

| Syntax | value_counts() 함수

```
데이터프레임['열 이름'].value_counts()
```

‘열 이름’ 열에서 값의 빈도수를 센다.

2. 데이터 전처리

■ 데이터 조작하기

▪ 값의 빈도수 구하기

- [코드 11-17]은 df 데이터프레임의 'result' 열에서 값의 빈도수를 구함

코드 11-17

```
01 print(df['result'].value_counts())
```

```
pass    3  
fail     2  
Name: result, dtype: int64
```

2. 데이터 전처리

■ 데이터 조작하기

▪ 값의 빈도수 구하기

- 그리고 [코드 11-18]은 df 데이터프레임의 'grade' 열의 각 값에 대한 빈도수를 구함

코드 11-18

```
01 print(df['grade'].value_counts())
```

```
B    2
C    2
A    1
Name: grade, dtype: int64
```

2. 데이터 전처리

■ 데이터 조작하기

■ 열 삭제하기

- drop 함수를 이용하면 데이터프레임에서 특정 열을 삭제할 수 있음

| Syntax | drop() 함수

```
데이터프레임.drop(columns='열 이름')
```

'열 이름' 열을 삭제한다. inplace=True 옵션을 추가하면 데이터프레임에 반영된다.

2. 데이터 전처리

■ 데이터 조작하기

▪ 열 삭제하기

- [코드 11-19]는 df 데이터프레임에서 'result' 열을 삭제함

코드 11-19

```
01 df.drop(columns='result', inplace=True)
02 display(df)
```

	name	exam	assign1	assign2	sum	grade
0	Jessica	45	20	19	84	B
1	Liam	30	17	14	61	C
2	Sophia	40	22	18	80	B
3	Ryan	37	18	15	70	C
4	Alex	48	24	25	97	A

```
df.drop(columns='result', inplace=True)
```

삭제할 열

삭제된 내용 반영

그림 11-12 데이터프레임에서 열을 삭제

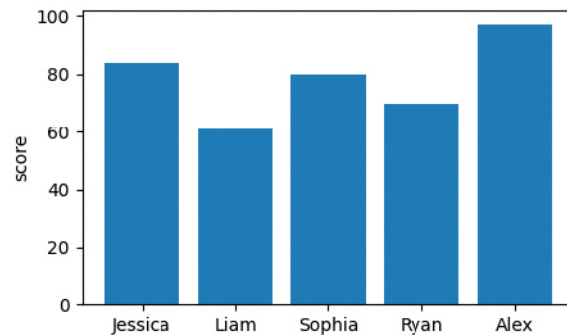
2. 데이터 전처리

■ 데이터프레임 시각화하기

- 모든 학생의 전체 성적을 막대 그래프로 나타내는 프로그램은 [코드11-20]과 같음

코드 11-20

```
01 import matplotlib.pyplot as plt
02
03 plt.figure(figsize=(5,3))
04 plt.bar(df['name'], df['sum'])
05 plt.ylabel('score')
06 plt.show()
```



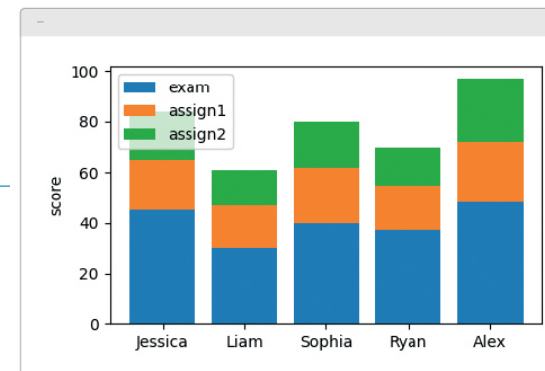
2. 데이터 전처리

■ 데이터프레임 시각화하기

- 각 학생의 시험과 과제 점수인 'test', 'assign1', 'assign2'를 누적 막대 그래프로 나타내는 프로그램은 [코드 11-21]과 같음

코드 11-21

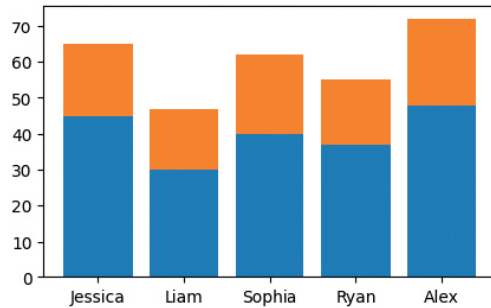
```
01 import matplotlib.pyplot as plt
02
03 plt.figure(figsize=(5,3))
04 plt.bar(df['name'], df['exam'], label='exam')
05 plt.bar(df['name'], df['assign1'], bottom=df['exam'], label='assign1')
06 plt.bar(df['name'], df['assign2'], bottom=df['exam']+df['assign1'], label='assign2')
07 plt.legend()
08 plt.ylabel('score')
09 plt.show()
```



2. 데이터 전처리

■ 데이터프레임 시각화하기

05행



06행

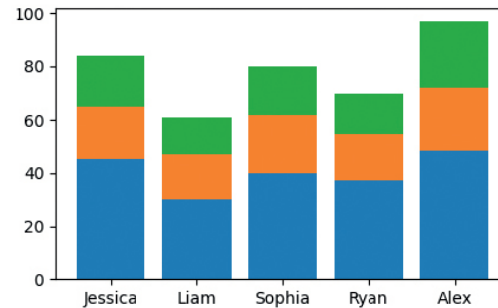


그림 11-13 누적 막대 그래프를 그리는 과정

2. 데이터 전처리

■ 누락된 데이터 처리하기

■ 누락된 데이터가 포함된 데이터프레임 생성하기

- 누락된 데이터를 넘파이 라이브러리의 nan으로 나타내기
- [코드 11-22]는 누락된 데이터가 포함된 df 데이터프레임을 생성하고 출력하는 예
- NaN은 'Not a Number'를 의미

코드 11-22

```
01 import numpy as np
02 import pandas as pd
03
04 df = pd.DataFrame({'name': ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex'],
05                   'gender': ['F', np.nan, np.nan, 'M', 'F'],
06                   'score': [89, 75, 95, np.nan, 93]})
07 display(df)
```

	name	gender	score
0	Jessica	F	89.0
1	Liam	NaN	75.0
2	Sophia	NaN	95.0
3	Ryan	M	NaN
4	Alex	F	93.0

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터 확인하기

- isna 함수를 이용하면 데이터프레임 안에 누락된 데이터가 있는지 확인할 수 있음

| Syntax | isna() 함수

```
import pandas as pd  
pd.isna(데이터프레임)
```

누락된 데이터가 있는지 확인한다. 실행하면 누락된 데이터는 True로 표시되고 아닌 데이터는 False로 표시된다.

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터 확인하기

- [코드 11-23]은 df 데이터프레임에 누락된 데이터가 포함되어 있는지를 확인함

코드 11-23

```
01 print(pd.isna(df))
```

```
name gender score
0  False  False  False
1  False   True  False
2  False   True  False
3  False  False   True
4  False  False  False
```

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터 확인하기

- 그런데 만약 데이터프레임 행이 많으면 이 방법으로 어느 데이터가 누락되었는지 찾기가 쉽지 않음
- isna 함수와 sum 함수를 함께 사용하면 각 열에 누락된 데이터가 몇 개씩 있는지 알 수 있음

코드 11-24

```
01 print(pd.isna(df).sum())
```

```
name      0
gender    2
score     1
dtype: int64
```

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터 삭제하기

- dropna 함수를 이용하면 누락된 데이터가 포함된 행 또는 열을 삭제할 수 있음

| Syntax | dropna() 함수

```
데이터프레임.dropna()
```

누락된 데이터가 포함된 행 또는 열을 삭제한다. 기본적으로 행을 삭제하고, `axis=1` 또는 `axis='columns'` 옵션을 추가하면 열을 삭제한다.

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터 삭제하기

- [코드 11-25]는 df 데이터프레임에서 누락된 데이터가 포함된 행을 제거함

코드 11-25

```
01 clean_df = df.dropna()  
02 display(clean_df)
```

	name	gender	score
0	Jessica	F	89.0
4	Alex	F	93.0

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터 삭제하기

- dropna 함수는 특정 행에서 데이터가 누락된 열이 하나라도 있으면 그 행 전체를 삭제함
- 그런데 'name'과 'score' 열만 분석한다면 'name'과 'score' 열이 누락된 행만 삭제해야 바람직
- 이런 경우에 subset 매개변수 값으로 'name'과 'score' 열을 지정함

2. 데이터 전처리

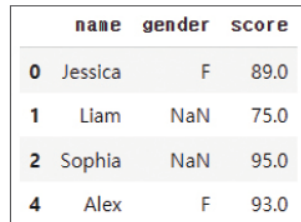
■ 누락된 데이터 처리하기

▪ 누락된 데이터 삭제하기

- [코드 11-26]의 실행 결과에서 'name' 열 또는 'score' 열에 누락된 데이터가 포함된 행만 삭제된 것을 확인할 수 있음

코드 11-26

```
01 clean_df = df.dropna(subset=['name', 'score'])
02 display(clean_df)
```



	name	gender	score
0	Jessica	F	89.0
1	Liam	NaN	75.0
2	Sophia	NaN	95.0
4	Alex	F	93.0

`df.dropna(subset=['name', 'score'])`

누락된 데이터를 찾을 열 이름

그림 11-14 특정 열의 데이터가 누락된 행만 삭제

2. 데이터 전처리

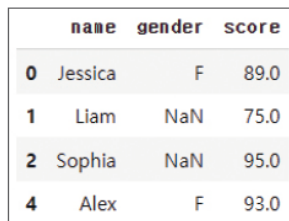
■ 누락된 데이터 처리하기

▪ 누락된 데이터 삭제하기

- dropna 함수를 이용해서 데이터가 누락된 행을 삭제하더라도 df 데이터프레임 원본에는 변화가 없음
- 만약 df에도 삭제된 내용이 반영되기를 원하면 [코드 11-27]과 같이 inplace=True 옵션을 추가

코드 11-27

```
01 df.dropna(subset=['name','score'], inplace=True)
02 display(df)
```



	name	gender	score
0	Jessica	F	89.0
1	Liam	NaN	75.0
2	Sophia	NaN	95.0
4	Alex	F	93.0

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터를 다른 값으로 대체하기

- fillna 함수를 이용하면 설정한 값으로 NaN을 대체할 수 있음

| Syntax | fillna() 함수

```
데이터프레임.fillna(값)
```

누락된 데이터를 '값'으로 대체한다.

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터를 다른 값으로 대체하기

- [코드 11-28]은 df 데이터프레임의 NaN을 0으로 대체하고 df1에 저장하는 코드인데, df 데이터프레임 원본에는 변화가 없음

코드 11-28

```
01 import pandas as pd
02
03 df = pd.DataFrame({'name': ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex'],
04                   'gender': ['F', np.nan, np.nan, 'M', 'F'],
05                   'score': [89, 75, 95, np.nan, 93]})
06
07 df1 = df.fillna(0)
08 display(df1)
```

	name	gender	score
0	Jessica	F	89.0
1	Liam	0	75.0
2	Sophia	0	95.0
3	Ryan	M	0.0
4	Alex	F	93.0

2. 데이터 전처리

■ 누락된 데이터 처리하기

■ 누락된 데이터를 다른 값으로 대체하기

- 대체할 값을 딕셔너리로 생성하고 fillna 함수에 입력하면 얼마다 다른 값으로 대체할 수 있음
- [코드 11-29]는 'gender' 열의 NaN을 'etc'로 대체하고 'score' 열의 NaN을 0으로 대체하는 코드

코드 11-29

```
01 df1 = df.fillna({'gender':'etc', 'score':0})  
02 display(df1)
```

	name	gender	score
0	Jessica	F	89.0
1	Liam	etc	75.0
2	Sophia	etc	95.0
3	Ryan	M	0.0
4	Alex	F	93.0

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터를 다른 값으로 대체하기

- 'score' 열의 누락된 데이터를 'score' 열 값의 평균으로 대체하려면 [코드 11-30]과 같이 mean 함수를 사용

코드 11-30

```
01 df1 = df.fillna({'gender':'etc', 'score':df['score'].mean()})  
02 display(df1)
```

	name	gender	score
0	Jessica	F	89.0
1	Liam	etc	75.0
2	Sophia	etc	95.0
3	Ryan	M	88.0
4	Alex	F	93.0

2. 데이터 전처리

■ 누락된 데이터 처리하기

▪ 누락된 데이터를 다른 값으로 대체하기

- 대체한 내용이 df에 반영되게 하려면 [코드 11-31]과 같이 fillna 함수에 inplace=True 옵션을 추가

코드 11-31

```
01 df.fillna({'gender':'etc', 'score':df['score'].mean()}, inplace=True)
02 display(df)
```

	name	gender	score
0	Jessica	F	89.0
1	Liam	etc	75.0
2	Sophia	etc	95.0
3	Ryan	M	88.0
4	Alex	F	93.0

2. 데이터 전처리

■ 데이터프레임 결합하기

- merge 함수를 이용하면 데이터프레임 두 개를 하나로 결합할 수 있음

| Syntax | merge() 함수

```
import pandas as pd  
pd.merge(데이터프레임1, 데이터프레임2)
```

데이터프레임 두 개를 하나로 결합한다.

2. 데이터 전처리

■ 데이터프레임 결합하기

- [코드 11-32]에서 두 개의 데이터프레임을 살펴보면 df1은 이름인 'name', 성별인 'gender', 나이인 'age' 열로 이루어지고, df2는 이름인 'name'과 소속 학과인 'department' 열로 이루어져 있음

df1				df2		
	name	gender	age		name	department
0	Jessica	F	21	0	Sophia	computer science
1	Liam	M	23	1	Liam	mathematics
2	Sophia	F	20	2	Ryan	law
3	Ryan	M	22	3	Alex	computer science
4	Alex	F	20	4	Jessica	law

2. 데이터 전처리

■ 데이터프레임 결합하기

코드 11-32

```
01 import pandas as pd
02
03 df1 = pd.DataFrame({'name': ['Jessica', 'Liam', 'Sophia', 'Ryan', 'Alex'],
04                     'gender': ['F', 'M', 'F', 'M', 'F'],
05                     'age': [21, 23, 20, 22, 20]})
06
07 df2 = pd.DataFrame({'name': ['Sophia', 'Liam', 'Ryan', 'Alex', 'Jessica'],
08                     'department': ['computer science', 'mathematics', 'law',
09                                     'computer science', 'law']})
```

2. 데이터 전처리

■ 데이터프레임 결합하기

- 데이터프레임을 하나로 결합하는 코드는 [코드 11-33]과 같음

코드 11-33

```
01 df3 = pd.merge(df1, df2)
02 display(df3)
```



	name	gender	age	department
0	Jessica	F	21	law
1	Liam	M	23	mathematics
2	Sophia	F	20	computer science
3	Ryan	M	22	law
4	Alex	F	20	computer science

2. 데이터 전처리

■ 데이터프레임 결합하기

- [코드 11-33]에서는 두 데이터프레임에 공통인 열 이름이 'name' 하나이므로 문제없이 동작함
- 공통인 열 이름이 여러 개 있을 때는 어떤 열을 기준으로 결합하는지 on 매개변수에 명시적으로 지정해야 함
- 사실 공통적인 열 이름이 하나일 때도 on 매개변수에 기준 열을 명시적으로 지정하는 것이 바람직
- 예를 들어 [코드 11-34]는 df1과 df2를 'name' 열로 결합하여 df3를 생성

코드 11-34

```
01 df3 = pd.merge(df1, df2, on='name')
```

2. 데이터 전처리

■ 데이터프레임 결합하기

- 추가로 학과명인 'department'와 학과장인 'head of department' 열로 이루어진 df4를 df3에 결합함
- df4 데이터프레임을 생성

df4

	department	head of department
0	computer science	Steve
1	mathematics	Emma
2	law	Carly

코드 11-35

```
01 df4 = pd.DataFrame({'department': ['computer science', 'mathematics', 'law'],  
02                        'head of department': ['Steve', 'Emma', 'Carly']})
```

2. 데이터 전처리

■ 데이터프레임 결합하기

- df3와 df4 데이터프레임의 결합 기준은 'department' 열이므로 [코드 11-36]과 같이 on 매개변수를 이용하여 df3와 df4를 결합

코드 11-36

```
01 df5 = pd.merge(df3, df4, on='department')
02 display(df5)
```

	name	gender	age	department	head of department
0	Jessica	F	21	law	Carly
1	Liam	M	23	mathematics	Emma
2	Sophia	F	20	computer science	Steve
3	Ryan	M	22	law	Carly
4	Alex	F	20	computer science	Steve

판다스 함수

3. 판다스 함수

■ 통계 함수

- 판다스에서는 데이터로부터 통계량을 구하는 다양한 함수를 제공

표 11-1 판다스 통계 함수

함수	설명	예시	예시 결과
count()	각 열에서 데이터 개수를 구한다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.count())</pre>	test 5 dtype: int64
sum()	각 열에서 데이터의 합을 구한다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.sum())</pre>	test 200 dtype: int64
cumsum()	각 열에서 데이터의 누적 합을 구한다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.cumsum())</pre>	test 0 45 1 75 2 115 3 152 4 200

3. 판다스 함수

■ 통계 함수

mean()	각 열에서 데이터의 평균을 구한다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.mean())</pre>	<pre>test 40.0 dtype: float64</pre>
std(), var()	각 열에서 데이터의 표준편차, 분산을 구한다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.std()) print(df.var())</pre>	<pre>test 7.035624 dtype: float64 test 49.5 dtype: float64</pre>
max(), min()	각 열에서 최댓값, 최솟값을 찾는다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.max()) print(df.min())</pre>	<pre>test 48 dtype: int64 test 30 dtype: int64</pre>
describe()	각 열에서 기초 통계량을 구한다.	<pre>import pandas as pd df = pd.DataFrame({'test': [45,30,40,37,48]}) print(df.describe())</pre>	<pre>test count 5.000000 mean 40.000000 std 7.035624 min 30.000000 25% 37.000000 50% 40.000000 75% 45.000000 max 48.000000</pre>

3. 판다스 함수

■ 통계 함수

- [코드 11-37]은 describe 함수를 이용해서 df 데이터프레임 각 열의 기본적인 통계를 구함

코드 11-37

```
01 import pandas as pd
02
03 df = pd.DataFrame({'test': [45, 30, 40, 37, 48],
04                   'assign1': [20, 17, 22, 18, 24],
05                   'assign2': [19, 14, 18, 15, 25]})
06 display(df)
07 print(df.describe())
```

3. 판다스 함수

■ 통계 함수

	test	assign1	assign2
0	45	20	19
1	30	17	14
2	40	22	18
3	37	18	15
4	48	24	25

	test	assign1	assign2
count	5.000000	5.000000	5.000000
mean	40.000000	20.200000	18.200000
std	7.035624	2.863564	4.32435
min	30.000000	17.000000	14.000000
25%	37.000000	18.000000	15.000000
50%	40.000000	20.000000	18.000000
75%	45.000000	22.000000	19.000000
max	48.000000	24.000000	25.000000

3. 판다스 함수

■ 문자열 함수

- 판다스에서 문자열을 다루는 대표적인 함수는 [표 11-2]와 같음

표 11-2 판다스 문자열 함수

함수	설명	예시	예시 결과								
find()	문자열에서 특정 문자열을 왼쪽부터 찾아서 인덱스를 반환한다. 찾은 문자열이 없으면 -1을 반환한다.	<pre>import pandas as pd df = pd.DataFrame({'높이':['1,250미터','740', '높이1,130미터']}) print(df['높이'].str.find('미터'))</pre>	<pre>0 5 1 -1 2 7 Name: 높이, dtype: int64</pre>								
strip()	문자열 양옆의 공백 또는 특정 문자를 제거한다.	<pre>import pandas as pd df = pd.DataFrame({'높이':['1,250미터','740', '높이1,130미터']}) df['높이'] = df['높이'].str.strip() display(df)</pre>	<table><thead><tr><th></th><th>높이</th></tr></thead><tbody><tr><td>0</td><td>1,250미터</td></tr><tr><td>1</td><td>740</td></tr><tr><td>2</td><td>높이1,130미터</td></tr></tbody></table>		높이	0	1,250미터	1	740	2	높이1,130미터
	높이										
0	1,250미터										
1	740										
2	높이1,130미터										
rstrip()	문자열 오른쪽의 공백 또는 특정 문자를 제거한다.	<pre>import pandas as pd df = pd.DataFrame({'높이':['1,250미터','740', '높이1,130미터']}) df['높이'] = df['높이'].str.rstrip('미터') display(df)</pre>	<table><thead><tr><th></th><th>높이</th></tr></thead><tbody><tr><td>0</td><td>1,250</td></tr><tr><td>1</td><td>740</td></tr><tr><td>2</td><td>높이1,130</td></tr></tbody></table>		높이	0	1,250	1	740	2	높이1,130
	높이										
0	1,250										
1	740										
2	높이1,130										

3. 판다스 함수

■ 문자열 함수

- 판다스에서 문자열을 다루는 대표적인 함수는 [표 11-2]와 같음

rstrip()	문자열 왼쪽의 공백 또는 특정 문자를 제거한다.	<pre>import pandas as pd df = pd.DataFrame({'높이':['1,250미터','740', '높이1,130미터']}) df['높이'] = df['높이'].str.strip().str. rstrip('높이') display(df)</pre>	<table><tr><th></th><th>높이</th></tr><tr><td>0</td><td>1,250미터</td></tr><tr><td>1</td><td>740</td></tr><tr><td>2</td><td>1,130미터</td></tr></table>		높이	0	1,250미터	1	740	2	1,130미터
	높이										
0	1,250미터										
1	740										
2	1,130미터										
replace()	특정 문자열을 다른 문자열로 교체한다.	<pre>import pandas as pd df = pd.DataFrame({'높이':['1,250미터','740', '높이1,130미터']}) df['높이'] = df['높이'].str.replace(',','') display(df)</pre>	<table><tr><th></th><th>높이</th></tr><tr><td>0</td><td>1250미터</td></tr><tr><td>1</td><td>740</td></tr><tr><td>2</td><td>높이1130미터</td></tr></table>		높이	0	1250미터	1	740	2	높이1130미터
	높이										
0	1250미터										
1	740										
2	높이1130미터										

3. 판다스 함수

■ 문자열 함수

- 데이터프레임 데이터의 자료형을 변환하는 함수는 `astype`

표 11-3 판다스 자료형 함수

함수	설명	예시	예시 결과
<code>astype()</code>	자료형을 변환한다.	<pre>import pandas as pd df = pd.DataFrame({'높이':['1250','740','1130']}) df['높이'] = df['높이'].astype(int) print(df['높이'])</pre>	<pre>0 1250 1 740 2 1130 Name: 높이, dtype: int64</pre>

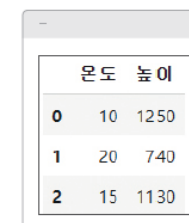
3. 판다스 함수

■ 문자열 함수

- 다음은 다양한 형식으로 표기된 온도와 높이에서 정수 부분만 추출하는 프로그램

코드 11-38

```
01 import pandas as pd
02
03 df = pd.DataFrame({'온도': ['섭씨10도', '20', '15도'],
04                    '높이': ['1,250미터', '740', '1,130미터']})
05 df['온도'] = df['온도'].str.strip() \
06             .str.rstrip('도') \
07             .str.lstrip('섭씨') \
08             .astype(int)
09 df['높이'] = df['높이'].str.strip() \
10             .str.rstrip('미터') \
11             .str.replace(',', '') \
12             .astype(int)
13 display(df)
```



	온도	높이
0	10	1250
1	20	740
2	15	1130

3. 판다스 함수

■ 문자열 함수

```
df['온도'] = df['온도'].str.strip() \ ① df에서 '온도' 열 문자열 데이터 양옆의 공백을 제거
        .str.rstrip('도') \          ② 문자열 데이터 오른쪽의 '도'를 제거
        .str.lstrip('섭씨') \        ③ 문자열 데이터 왼쪽의 '섭씨'를 제거
        .astype(int)                 ④ 정수로 변환
```

그림 11-15 온도 데이터를 정수로 변환

```
df['높이'] = df['높이'].str.strip() \ ① df에서 '높이' 열 문자열 데이터 양옆의 공백을 제거
        .str.rstrip('미터') \        ② 문자열 데이터 오른쪽의 '미터'를 제거
        .str.replace(',', '') \      ③ 문자열 데이터의 ','를 ''로 변경
        .astype(int)                 ④ 정수로 변환
```

그림 11-16 높이 데이터를 정수로 변환

3. 판다스 함수

LAB 올림픽 메달 수 그래프 그리기

- 4개국의 메달 수 데이터를 데이터프레임으로 저장하고 누적 막대 그래프로 나타내기

표 11-4 1988 서울 올림픽에서 4개국이 획득한 메달 수 © Olympics

국가	금	은	동
대한민국	12	10	11
미국	36	31	27
일본	4	3	7
프랑스	6	4	6

3. 판다스 함수

LAB 올림픽 메달 수 그래프 그리기

- 4개국의 메달 수 데이터를 데이터프레임으로 저장하고 누적 막대 그래프로 나타내기

표 11-4 1988 서울 올림픽에서 4개국이 획득한 메달 수 © Olympics

국가	금	은	동
대한민국	12	10	11
미국	36	31	27
일본	4	3	7
프랑스	6	4	6

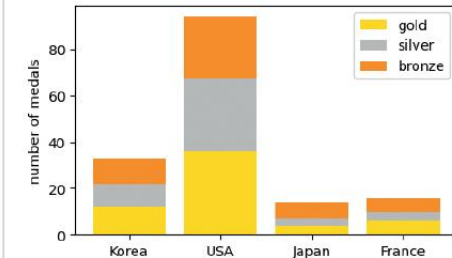
3. 판다스 함수

LAB 올림픽 메달 수 그래프 그리기

코드 11-39

```
01 import pandas as pd
02 import matplotlib.pyplot as plt
03
04 df = pd.DataFrame({'nation': ['Korea', 'USA', 'Japan', 'France'],
05                   'gold': [12, 36, 4, 6],
06                   'silver': [10, 31, 3, 4],
07                   'bronze': [11, 27, 7, 6]})
08 display(df)
09
10 plt.figure(figsize=(5,3))
11 plt.bar(df['nation'], df['gold'], color='gold', label='gold')
12 plt.bar(df['nation'], df['silver'], color='silver', bottom=df['gold'], label='silver')
13 plt.bar(df['nation'], df['bronze'], color='darkorange',
14         bottom=df['gold']+df['silver'], label='bronze')
15 plt.legend()
16 plt.ylabel('number of medals')
17 plt.show()
```

	nation	gold	silver	bronze
0	Korea	12	10	11
1	USA	36	31	27
2	Japan	4	3	7
3	France	6	4	6



파일 데이터 이용

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ CSV 파일 생성하기

- CSV(comma-separated values) 파일은 값을 콤마(,)로 구분한 텍스트 파일이며 대부분의 프로그램에서 CSV 파일을 열 수 있음
- 또한 CSV 파일은 엑셀 파일에 비해 용량이 작아 프로그래밍할 때 가장 많이 사용됨
- 다음 2017년부터 2021년까지의 계절별 강수량 데이터를 CSV 파일로 만들어보기

표 11-5 2017년~2021년까지의 계절별 강수량 © 지표누리

연도	봄	여름	가을	겨울
2017	124.9	612.7	177.9	75.2
2018	383.5	620.6	351.3	68.7
2019	175.0	508.2	440.8	168.8
2020	173.7	1037.6	270.4	47.8
2021	330.5	612.8	256.4	13.3

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ CSV 파일 생성하기

1. 메모장 프로그램에서 다음과 같이 [표 11-5]의 내용을 입력. 콤마(,)와 값 사이에 공백이 없어야 함

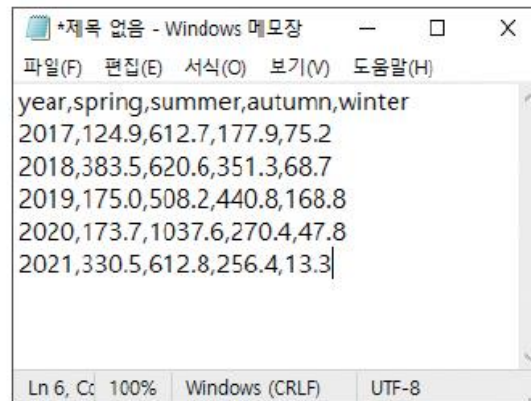


그림 11-17 메모장에 데이터 입력

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ CSV 파일 생성하기

2. [파일] 메뉴의 [저장]을 선택
3. '다른 이름으로 저장' 대화상자 아래쪽에서 [파일 형식]을 '모든 파일'로 설정하고 [파일 이름]을 'precipitation.csv'로 입력

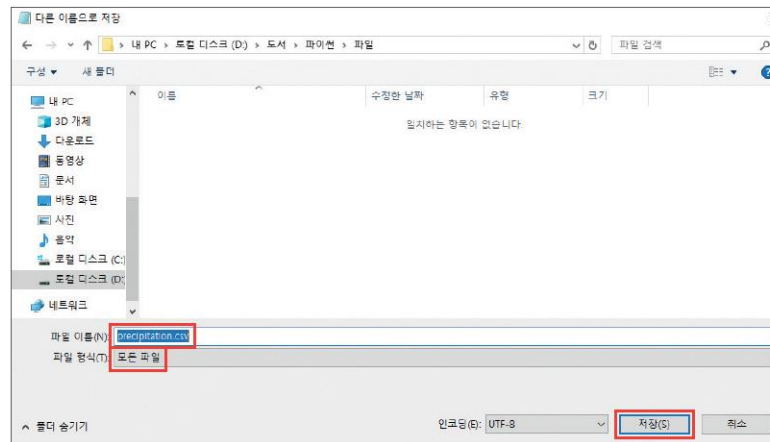


그림 11-18 CSV 파일로 저장

4. [저장] 버튼을 눌러 생성한 데이터 파일을 CSV 파일로 저장

4. 파일 데이터 이용

■ 계절별 강수량 데이터

▪ CSV 파일 불러오기

- 앞에서 생성한 'precipitation.csv' 파일을 코랩 세션 저장소에 업로드

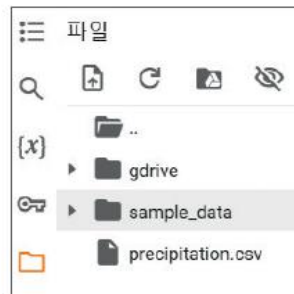


그림 11-19 코랩 세션 저장소에 CSV 파일 업로드

4. 파일 데이터 이용

■ 계절별 강수량 데이터

▪ CSV 파일 불러오기

- CSV 파일을 읽어들이기 때에는 판다스의 read_csv 함수를 사용

| Syntax | read_csv() 함수

```
import pandas as pd  
pd.read_csv(CSV파일)
```

'CSV파일'을 데이터프레임 형식으로 읽어온다.

4. 파일 데이터 이용

■ 계절별 강수량 데이터

▪ CSV 파일 불러오기

- [코드 11-40]은 precipitation.csv 파일을 읽어와서 데이터프레임 형식으로 df에 저장한 후 출력

코드 11-40

```
01 import numpy as np
02 import pandas as pd
03 import matplotlib.pyplot as plt
04
05 df = pd.read_csv('precipitation.csv')
06 display(df)
```

	year	spring	summer	autumn	winter
0	2017	124.9	612.7	177.9	75.2
1	2018	383.5	620.6	351.3	68.7
2	2019	175.0	508.2	440.8	168.8
3	2020	173.7	1037.6	270.4	47.8
4	2021	330.5	612.8	256.4	13.3

4. 파일 데이터 이용

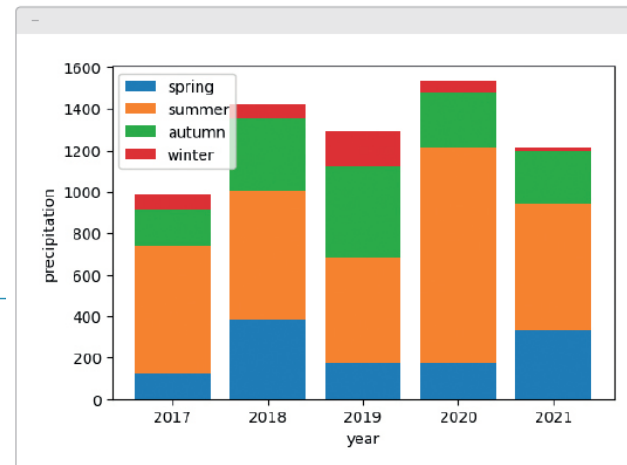
■ 계절별 강수량 데이터

■ 시각화 및 데이터 분석하기

- [코드 11-41]은 계절별 강수량을 누적 막대 그래프로 나타냄

코드 11-41

```
01 plt.figure(figsize=(6,4))
02 plt.bar(df['year'], df['spring'], label='spring')
03 plt.bar(df['year'], df['summer'], bottom=df['spring'], label='summer')
04 plt.bar(df['year'], df['autumn'], bottom=(df['spring']+df['summer']), label='autumn')
05 plt.bar(df['year'], df['winter'], bottom=(df['spring']+df['summer']+df['autumn']),
06         label='winter')
07 plt.ylabel('precipitation')
08 plt.xlabel('year')
09 plt.legend()
```



4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ 시각화 및 데이터 분석하기

- [코드 11-42]는 계절별 강수량을 더해 연도별 강수량을 구함

코드 11-42

```
01 df['sum'] = df['spring']+df['summer']+df['autumn']+df['winter']  
02 display(df)
```

	year	spring	summer	autumn	winter	sum
0	2017	124.9	612.7	177.9	75.2	990.7
1	2018	383.5	620.6	351.3	68.7	1424.1
2	2019	175.0	508.2	440.8	168.8	1292.8
3	2020	173.7	1037.6	270.4	47.8	1529.5
4	2021	330.5	612.8	256.4	13.3	1213.0

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ 데이터프레임을 CSV 파일로 저장하기

- 데이터프레임을 CSV 파일로 저장할 때 판다스의 to_csv 함수를 사용

| Syntax | to_csv() 함수

```
데이터프레임.to_csv(CSV파일이름)
```

데이터프레임을 'CSV파일이름' 파일로 저장한다.

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ 데이터프레임을 CSV 파일로 저장하기

- 다음은 'sum' 열이 추가된 df 데이터프레임을 precipitation_out.csv라는 파일로 저장하는 코드
- 코드를 실행하면 코랩 세션 저장소에 precipitation_out.csv 파일이 생성됨

코드 11-43

```
01 df.to_csv('precipitation_out.csv')
```

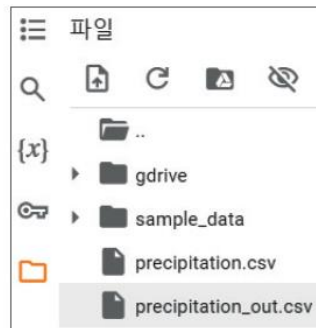


그림 11-20 코랩 세션 저장소에 CSV 파일 생성

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ 데이터프레임을 CSV 파일로 저장하기

- precipitation_out.csv 파일을 다운로드하고 엑셀이나 구글 스프레드시트에서 열어보면 행 인덱스 번호를 확인할 수 있음

	year	spring	summer	autumn	winter	sum
0	2017	124.9	612.7	177.9	75.2	990.7
1	2018	383.5	620.6	351.3	68.7	1424.1
2	2019	175	508.2	440.8	168.8	1292.8
3	2020	173.7	1037.6	270.4	47.8	1529.5
4	2021	330.5	612.8	256.4	13.3	1213

행 인덱스 번호

그림 11-21 엑셀에서 행 인덱스 번호 확인

4. 파일 데이터 이용

■ 계절별 강수량 데이터

■ 데이터프레임을 CSV 파일로 저장하기

- 행 인덱스 번호가 포함되지 않게 하려면 다음과 같이 `index=False` 옵션을 추가함

코드 11-44

```
01 df.to_csv('precipitation_out.csv', index=False)
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

1. 국가통계포털(<https://kosis.kr>)에 접속해서 화면 왼쪽 위의 [국내통계] 메뉴를 선택

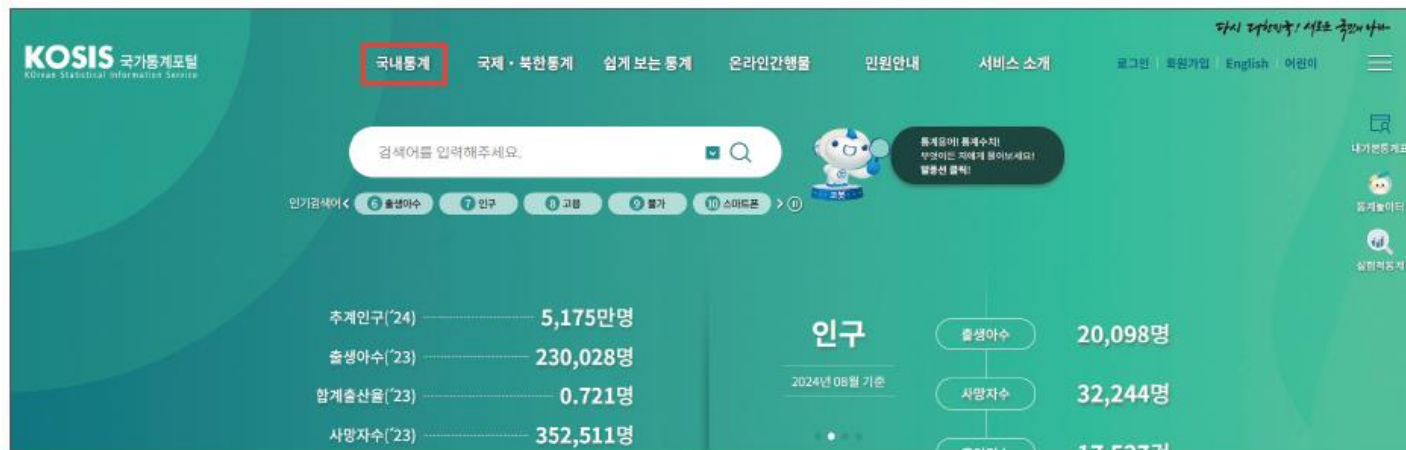


그림 11-22 국가통계포털

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

2. [e-지방지표(통계표)]를 선택

국내통계	국제 · 북한통계	쉽게 보는 통계	온라인간행물	민원안내	서비스 소개
주제별 통계	국제통계	대상별 접근	주제별	FAQ	국가통계포털 소개
기관별 통계	세계속의 한국 	이슈별 접근	명칭별	Q&A	국가통계현황
e-지방지표(통계표)	국제지역통계	통계시각화콘텐츠	기획간행물	KOSIS 길라잡이	국가통계 공표일정
e-지방지표(시각화) 	북한통계			홈페이지 개선의견	새소식
과거 · 중지통계				찾아가는 KOSIS	팩트체크 서비스
실험적통계 					서비스정책
					공유서비스(OpenAPI)

그림 11-23 e-지방지표

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

3. [교육] 주제의 [학급당 학생수(시도/시/군/구)]를 선택



그림 11-24 학급당 학생 수 데이터

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

4. 화면 오른쪽 위의 [조회설정] 버튼을 선택하면 조건을 설정하여 일부 데이터만 볼 수 있음. [항목]은 '전체'를 선택하고, [행정구역별]은 '서울특별시'와 '제주특별자치도'를 선택. 그리고 [시점]은 2011~2024를 선택한 후 [조회] 버튼을 클릭

KOSIS

통계목록 : e-자랑자료(주제별) >

1. 학급당 학생수(시도/시/군/구) X 모두 보기

한국교육개발원 (자료문의서: 02-2012-9114 / 043-530-9620)

수색기간 : 년 2011 ~ 2024 / 자료갱신일: 2024-09-27 / 주석정보

[시점] [항목] [행정/중간/광역] [연고정액제] [세입액] [현역조사] [주수/중세] [스크린] [OPENAPI] [인쇄] [다운로드] [조회설정]

(단위: 명)

행정구역별	2024				
	전체	유치원	초등학교	중학교	고등학교
전국	21.11	15.72	19.99	24.53	23.36
서울특별시	21.77	17.09	20.83	23.77	23.65
부산광역시	21.33	18.06	20.93	24.41	21.07
대구광역시	22.21	18.73	21.71	24.82	22.82
인천광역시	22.18	17.37	21.23	26.18	23.47
광주광역시	21.04	16.72	19.79	23.60	22.97
대전광역시	20.24	15.53	19.43	23.97	21.50
울산광역시	21.84	17.20	20.89	24.88	23.59
세종특별자치시	20.50	13.63	19.85	23.93	23.55

(a) 조회설정 버튼

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

조회조건 부가기능

Q 조회

항목

검색하기 검색

☐ 전체선택

☒ 전체

☐ 유치원

☐ 초등학교

☐ 중학교

☐ 고등학교

행정구역별

시점

(b) 조회할 항목 설정

조회조건 부가기능

Q 조회

항목

행정구역별

검색하기 검색

(기본값)하위옵션 선택안함

레벨선택 전체해제 + 펼치기

☐ 전국

☒ 서울특별시 1

☐ 부산광역시

☐ 대구광역시

☐ 인천광역시

☐ 광주광역시

☐ 대전광역시

☐ 경상북도

☐ 경상남도

☒ 제주특별자치도 2

(c) 조회할 행정구역 설정

조회조건 부가기능

2 Q 조회

항목

행정구역별

시점

년 1

2011 - 2024

☒ 전체선택

☒ 2024

☒ 2023

☒ 2022

☒ 2021

☒ 2020

☒ 2019

☒ 2018

☒ 2017

☒ 2016

☒ 2015

(d) 조회할 시점 설정

그림 11-25 조회 조건 설정

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

5. 조건에 맞는 데이터가 표로 나타남

1) 학급당 학생수(시도/시/군/구)								
수록기간 : 년 2011 ~ 2024 / 자료경신일: 2024-09-27 / 주석정보								
시점 증감/증감률 행정전환 열고정예제 새 탭 열기 화면복사 주소/출처								
(단위: 명)								
행정구역별	2011	2012	2013	2014	2015	2016	2017	2018
	전체	전체	전체	전체	전체	전체	전체	전체
	▲ ▼ □	▲ ▼ □	▲ ▼ □	▲ ▼ □	▲ ▼ □	▲ ▼ □	▲ ▼ □	▲ ▼ □
서울특별시	29.36	28.63	27.81	26.96	26.19	25.36	24.60	23.76
제주특별자치도	28.88	28.09	27.25	26.46	25.97	25.53	25.25	24.97

그림 11-26 데이터 조회 결과

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기
6. 지금은 '행정구역'이 행에 있고 '연도'가 열에 있지만 '행정구역'이 열에 있어야 파이썬에서 사용하기 수월함. [행렬전환] 버튼을 클릭한 다음 [그림 11-27]과 같이 '시점', '항목'과 '행정구역별'의 위치를 전환한 다음 [적용] 버튼을 클릭

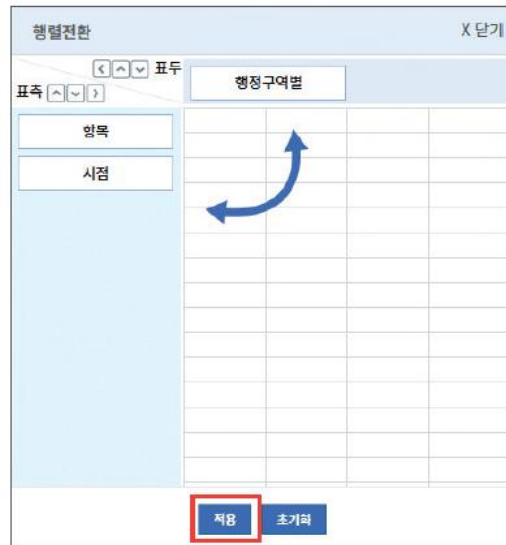


그림 11-27 행렬전환

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

7. 행과 열이 전환되어 '행정구역'이 열에 있는 것을 확인하고 [다운로드] 버튼을 클릭

시점	서울특별시	제주특별자치도
2011	29.36	28.88
2012	28.63	28.09
2013	27.81	27.26
2014	26.96	26.46
2015	26.19	25.97
2016	25.36	25.53
2017	24.60	25.25
2018	23.76	24.97
2019	22.96	24.47
2020	22.45	24.15
2021	22.47	23.70
2022	22.03	23.36
2023	21.84	22.72
2024	21.77	22.36

그림 11-28 데이터 다운로드

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 학급당 학생 수 데이터 수집하기

- 서울과 제주의 학급당 학생 수 데이터를 수집해 보기

7. 다운로드 창이 열리면 [파일형태]를 'CSV'로 선택하고 [다운로드] 버튼을 클릭.
'학급당학생수.csv' 파일이 저장됨



The screenshot shows a '다운로드' (Download) window with a close button 'X 닫기' in the top right. Below the title bar is a '메타데이터보기 (TXT)' button. The main section is titled '파일형태' (File Format) and includes several options: 'EXCEL(xlsx)', 'EXCEL(xls) (셀 병합)', 'CSV (인코딩: ANSI)' (selected), 'TXT (인코딩: ANSI)', and 'SDMX(2.0)' (with sub-options 'DSD (데이터구조)' and 'DATA Generic'). There are also checkboxes for '빈셀 부호(-)' (checked), '통계부호', and '코드포함'. Below this is a '시점정렬' (Time Alignment) section with '오늘자순' (selected) and '내림자순'. The '소수점' (Decimal) section has '수목자료형식과 동일' and '조회화면과 동일' (selected). At the bottom, a red box highlights the '다운로드' button.

그림 11-29 다운로드 창

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ CSV 파일 불러오기

- 다운로드한 '학급당학생수.csv' 파일을 코랩 세션 저장소에 업로드

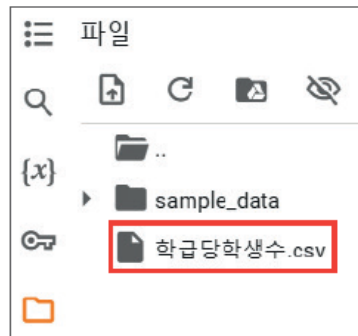


그림 11-30 코랩 세션 저장소에 CSV 파일 업로드

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ CSV 파일 불러오기

- [코드 11-45]는 학급당학생수.csv 파일을 읽어와서 df에 저장하는데, 실행해 보면 오류 메시지가 출력됨

코드 11-45

```
01 import numpy as np
02 import pandas as pd
03 import matplotlib.pyplot as plt
04
05 df = pd.read_csv('학급당학생수.csv')
06 display(df)
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ CSV 파일 불러오기

```
-----  
UnicodeDecodeError                                Traceback (most recent call last)  
<ipython-input-1-3a5c477c15e5> in <cell line: 5>()  
      3 import matplotlib.pyplot as plt  
      4  
----> 5 df = pd.read_csv('학급당학생수.csv')  
      6 display(df)  
  
/usr/local/lib/python3.10/dist-packages/pandas/_libs/parsers.pyx in pandas._libs.parsers.  
raise_parser_error()  
  
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc7 in position 1: invalid  
continuation byte
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ CSV 파일 불러오기

- UnicodeDecodeError 오류는 파일이 'utf-8' 형식이 아닐 때 발생
- 한글 인코딩 방식은 크게 UTF-8과 EUC-KR로 구분할 수 있는데, '학급당학생수.csv' 파일의 한글 인코딩 방식 이 EUC-KR이라서 오류가 발생
- 이런 경우 [그림 11-31]과 같이 read_csv 함수에 encoding='EUC-KR' 옵션을 추가하면 대부분 해결됨

```
df = pd.read_csv('학급당학생수.csv', encoding='EUC-KR')
```

한글 인코딩 방식이 EUC-KR

그림 11-31 한글 인코딩 방식을 설정하는 코드

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ CSV 파일 불러오기

코드 11-46

```
01 df = pd.read_csv('학급당학생수.csv', encoding='EUC-KR')  
02 display(df)
```

	항목	시점	서울특별시	제주특별자치도
0	전체	2011	29.36	28.88
1	전체	2012	28.63	28.09
2	전체	2013	27.81	27.26
3	전체	2014	26.96	26.46
4	전체	2015	26.19	25.97
5	전체	2016	25.36	25.53
6	전체	2017	24.60	25.25
7	전체	2018	23.76	24.97
8	전체	2019	22.96	24.47
9	전체	2020	22.45	24.15
10	전체	2021	22.47	23.70
11	전체	2022	22.03	23.36
12	전체	2023	21.84	22.72
13	전체	2024	21.77	22.36

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ df 데이터프레임 파악하기

- df 데이터프레임의 크기가 클 때 모든 내용을 출력하는 것은 바람직하지 않음
- head와 tail 함수를 이용하면 데이터프레임의 앞부분 또는 뒷부분을 대표로 확인할 수 있음

| Syntax | head() 함수, tail() 함수

```
데이터프레임.head()
```

데이터프레임의 앞부분을 확인한다.

```
데이터프레임.tail()
```

데이터프레임의 뒷부분을 확인한다.

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ df 데이터프레임 파악하기

- [코드 11-47]은 head 함수를 이용해서 df 데이터프레임의 앞부분을 출력하고, tail 함수를 이용해서 df 데이터프레임의 뒷부분을 출력하는 코드
- head 함수는 df의 첫 번째 행부터 다섯 번째 행까지 출력하고, tail 함수는 df의 뒤에서 다섯 번째 행부터 마지막 행까지 출력함

코드 11-47

```
01 display(df.head())  
02 display(df.tail())
```

	항목	시점	서울특별시	제주특별자치도
0	전체	2011	29.36	28.88
1	전체	2012	28.63	28.09
2	전체	2013	27.81	27.26
3	전체	2014	26.96	26.46
4	전체	2015	26.19	25.97
	항목	시점	서울특별시	제주특별자치도
9	전체	2020	22.45	24.15
10	전체	2021	22.47	23.70
11	전체	2022	22.03	23.36
12	전체	2023	21.84	22.72
13	전체	2024	21.77	22.36

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ df 데이터프레임 파악하기

- shape 속성을 이용하면 데이터프레임의 행과 열의 크기를 알 수 있음

| Syntax | shape 속성

```
데이터프레임.shape
```

'데이터프레임'의 행과 열의 크기를 알려준다.

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ df 데이터프레임 파악하기

- [코드 11-48]은 df 데이터프레임의 행과 열의 크기를 확인하는 코드이고, 실행 결과 (14, 4)는 14행 4열을 의미

코드 11-48

```
01 print(df.shape)
```

```
(14, 4)
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 그래프에 한글 표시하기

- 한글을 표시하려면 [코드 11-49]와 같이 한글 폰트를 설치해야 함

코드 11-49

```
01 sudo apt-get install -y fonts-nanum
02 sudo fc-cache -fv
03 rm -rf ~/.cache/matplotlib
```

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  fonts-nanum
0 upgraded, 1 n
:
/var/cache/fontconfig: cleaning cache directory
/root/.cache/fontconfig: not cleaning non-existent cache directory
/root/.fontconfig: not cleaning non-existent cache directory
fc-cache: succeeded
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

▪ 그래프에 한글 표시하기

- [코드 11-50]은 font10 변수를 생성하고 크기가 10인 나눔고딕체의 속성 정보를 저장

코드 11-50

```
01 path = '/usr/share/fonts/truetype/nanum/NanumGothicBold.ttf'  
02 import matplotlib.font_manager as fm  
03 font10 = fm.FontProperties(fname=path, size=10)
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 그래프에 한글 표시하기

- [코드 11-51]은 연도별 학급당 학생수를 선 그래프로 나타냅니다. 축 제목이 한글로 잘 출력된 것을 확인할 수 있음
- 한글을 출력하는 함수 안에 fontproperties=font10 옵션을 추가하면 되는데, font10에는 한글 폰트 속성 정보가 저장되어 있음

코드 11-51

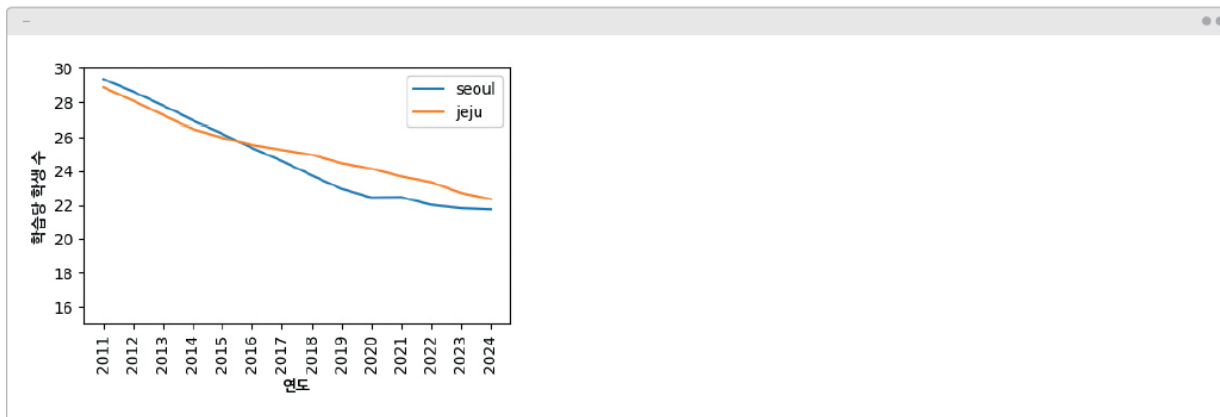
```
01 ind = np.arange(len(df))
02 plt.figure(figsize=(5,3))
03 plt.plot(ind, df['서울특별시'], label='seoul')
04 plt.plot(ind, df['제주특별자치도'], label='jeju')
05 plt.xticks(ind, df['시점'])
```

4. 파일 데이터 이용

■ 학급당 학생 수 데이터

■ 그래프에 한글 표시하기

```
06 plt.ylim(15, 30)
07 plt.legend()
08 plt.xlabel('연도', fontproperties=font10)
09 plt.ylabel('학급당 학생 수', fontproperties=font10)
10 plt.xticks(rotation='vertical')
11 plt.show()
```



4. 파일 데이터 이용

LAB 지역별 면적과 인구 분석

- CSV 파일을 이용해서 우리나라 지역별 인구밀도를 구하고 그래프로 표현해 보기

표 11-6 2021년 기준 지역별 면적 © 통계청

지역	면적	지역	면적	지역	면적
서울특별시	605	울산광역시	1,062	전라북도	8,072
부산광역시	770	세종특별자치시	465	전라남도	12,359
대구광역시	884	경기도	10,197	경상북도	19,035
인천광역시	1,066	강원도	16,830	경상남도	10,542
광주광역시	501	충청북도	7,407	제주특별자치도	1,850
대전광역시	540	충청남도	8,247		

4. 파일 데이터 이용

LAB 지역별 면적과 인구 분석

표 11-7 2022년 기준 지역별 인구수 © 통계청

지역	인구 수	지역	인구 수	지역	인구 수
서울특별시	9,428,372	울산광역시	1,110,663	전라북도	1,769,607
부산광역시	3,317,812	세종특별자치시	383,591	전라남도	1,817,697
대구광역시	2,363,691	경기도	13,589,432	경상북도	2,600,492
인천광역시	2,967,314	강원도	1,536,498	경상남도	3,280,493
광주광역시	1,431,050	충청북도	1,595,058	제주특별자치도	678,159
대전광역시	1,446,072	충청남도	2,123,037		

4. 파일 데이터 이용

LAB 지역별 면적과 인구 분석

- '지역별면적.csv'와 '지역별인구수.csv' 파일을 코랩 세션 저장소에 업로드
- 역별 인구밀도를 그래프로 나타내는 프로그램은 다음과 같음

코드 11-52

```
01 !sudo apt-get install -y fonts-nanum
02 !sudo fc-cache -fv
03 !rm -rf ~/.cache/matplotlib
04 path = '/usr/share/fonts/truetype/nanum/NanumGothicBold.ttf'
05 import matplotlib.font_manager as fm
06 font9 = fm.FontProperties(fname=path, size=9)
07 font11 = fm.FontProperties(fname=path, size=11)
08 import numpy as np
09 import pandas as pd
10 import matplotlib.pyplot as plt
11
12 df1 = pd.read_csv('지역별면적.csv', encoding='EUC-KR')
13 df2 = pd.read_csv('지역별인구수.csv', encoding='EUC-KR')
```


4. 파일 데이터 이용

LAB 지역별 면적과 인구 분석

```
14
15 df = pd.merge(df1, df2, on='지역')
16
17 df['인구밀도'] = df['총인구수'] / df['면적']
18 display(df[df['인구밀도']>=2000])
19 ind = np.arange(len(df))
20 plt.figure(figsize=(5,3))
21 plt.bar(ind, df['인구밀도'])

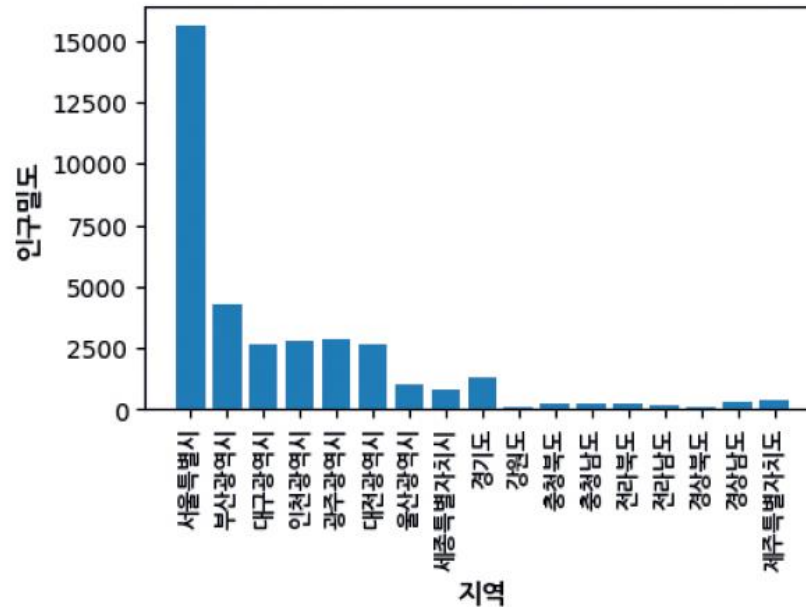
22 plt.xlabel('지역', fontproperties=font11)
23 plt.ylabel('인구밀도', fontproperties=font11)
24 plt.xticks(ind, df['지역'], rotation='vertical', fontproperties=font9)
25 plt.show()
```

4. 파일 데이터 이용

LAB 지역별 면적과 인구 분석

	지역	면적	총인구수	인구밀도
0	서울특별시	605	9428372	15584.085950
1	부산광역시	770	3317812	4308.846753
2	대구광역시	884	2363691	2673.858597
3	인천광역시	1066	2967314	2783.596623
4	광주광역시	501	1431050	2856.387226

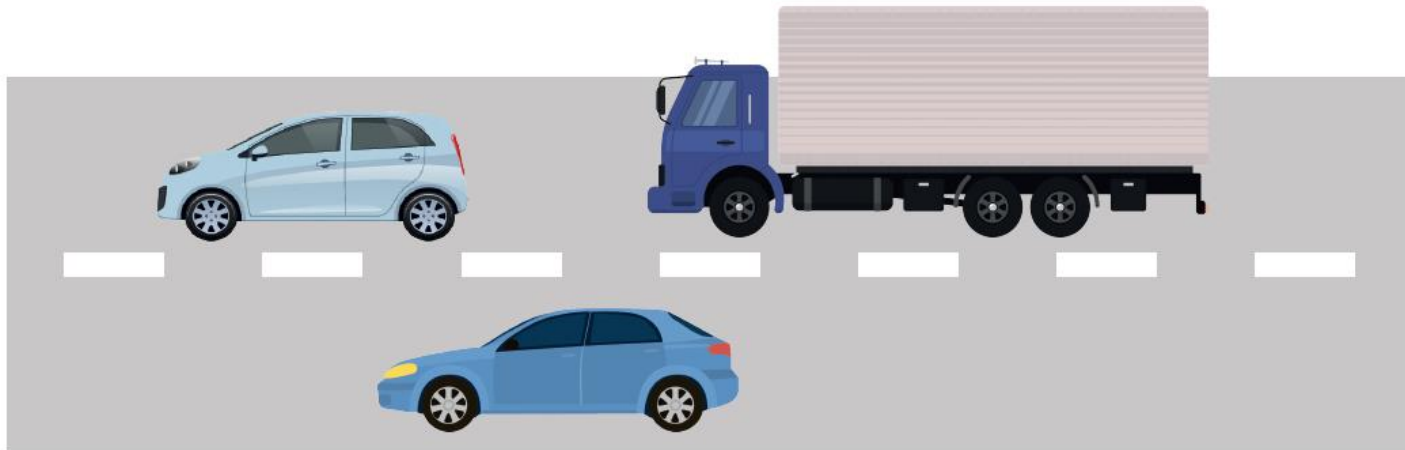
	지역	면적	총인구수	인구밀도
0	서울특별시	605	9428372	15584.085950
1	부산광역시	770	3317812	4308.846753
2	대구광역시	884	2363691	2673.858597
3	인천광역시	1066	2967314	2783.596623
4	광주광역시	501	1431050	2856.387226
5	대전광역시	540	1446072	2677.911111



[플러스 예제] 자동차 배기량 과 CO₂ 배출량의 상관관계

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

- 휘발유 차량과 경유 차량에 대해 자동차 배기량과 CO₂ 배출량의 상관관계를 분석해 보기



5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 데이터 수집하기

- 공공데이터포털(<https://www.data.go.kr>)에 접속해서 '자동차 연비표시제도'를 검색



그림 11-32 공공데이터포털 검색창

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 데이터 수집하기

- 검색 결과 화면에서 '한국에너지공단_자동차 표시연비 정보'를 찾고 오른쪽의 [다운로드] 버튼을 클릭하여 'car20240731.csv'를 다운로드

파일데이터 (1건) [더보기 >](#)

산업고용

공공기관

[미리보기](#)

CSV

JSON + XML

한국에너지공단_자동차 표시연비 정보

소비자가 우수한 연비의 자동차를 구매할 수 있도록 자동차의 에너지소비효율 및 등급 정보를 제공하여 국내 판매 승용차의 연비향상을 통한 에너지절약을 도모합니다.

제공기관 한국에너지공단 수정일 2024-08-27 조회수 16478 다운로드 4178 주기성 데이터 6 키워드 도심연비,고속연비,배기가스

[다운로드](#)

오픈 API (1건) [더보기 >](#)

산업고용

공공기관

[미리보기](#)

XML

JSON

한국에너지공단_자동차 에너지효율등급 정보

업체명과 모델명, 효율등급을 이용해 자동차 에너지효율등급에 대한 신청번호, 업체명, 모델명, 배기량, 연비, 등급, 연간 유류비 정보 조회 서비스

제공기관 한국에너지공단 수정일 2022-06-14 조회수 4436 활용신청 54 키워드 자동차,에너지효율,등급

[활용신청](#)

그림 11-33 CSV 파일 다운로드

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ CSV 파일 불러오기

- 'car20240731.csv' 파일을 코랩 세션 저장소에 업로드

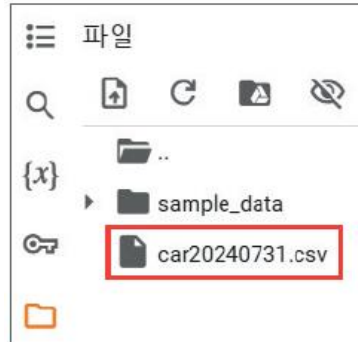


그림 11-34 CSV 파일 업로드

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ CSV 파일 불러오기

- 넘파이, 판다스, 맷플롯립 라이브러리를 불러옴

```
01 import numpy as np
02 import pandas as pd
03 import matplotlib.pyplot as plt
```


5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ CSV 파일 불러오기

- 'car20240731.csv' 파일을 읽어와서 df에 저장함
- df의 크기를 출력하고 앞부분 데이터를 확인

```
01 df = pd.read_csv('car20240731.csv')
02 print(len(df))
03 display(df.head())
```

3498

업체명	모델명	연료	변속형식	차량형식	자동차 종류	도심주행연비	고속도로연비	CO2배출량 (g_kn)	1회충전주행거리 (km)	배기량	등급	출시연도	
0	볼보	볼보 V60CCB5 AWD	휘발유	자동	하이브리드	승용차	8.7	11.9	175.0	NaN	1969.0	4등급	2024
1	볼보	볼보 XC60T8 AWD	전기+휘발유	자동	PHEV	승용차	16.6	17.1	33.0	61.0	1969.0	PHEV	2024
2	벤츠	Mercedes-Benz GLC 220 d 4MATIC	경유	자동	하이브리드	승용차	13.8	15.5	130.0	NaN	1993.0	2등급	2024
3	벤츠	Mercedes-Benz E 450 4MATIC	휘발유	자동	하이브리드	승용차	9.0	13.1	162.0	NaN	2998.0	4등급	2024
4	랜드로버	더 뉴 레인지로버 P550e SWB	전기+휘발유	자동	PHEV	승용차	11.5	13.1	35.0	80.0	2996.0	PHEV	2024

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 데이터 전처리하기

- df 데이터프레임에서 각 열의 기본 정보를 확인
- '고속도로연비'와 '배기량' 열의 기초 통계량을 확인해 보면 두 열 모두 최솟값이 0인 데이터가 포함되어 있음

```
01 display(df.describe())
```

	도심주행연비	고속도로연비	CO2배출량(g_km)	1회충전주행거리(km)	배기량	출시연도
count	3497.000000	3497.000000	3186.000000	403.000000	3187.000000	3498.000000
mean	9.620703	11.752617	168.908035	265.257816	2557.875118	2020.774728
std	4.493003	4.508477	48.694492	162.490485	965.196309	2.187160
min	1.600000	0.000000	14.000000	11.000000	0.000000	2008.000000
25%	7.300000	9.800000	137.000000	69.000000	1995.000000	2019.000000
50%	9.200000	11.800000	167.000000	303.000000	2199.000000	2021.000000
75%	11.500000	14.100000	199.000000	408.000000	2997.000000	2023.000000
max	99.500000	92.600000	354.000000	555.000000	6749.000000	2024.000000

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 데이터 전처리하기

- 배기량과 CO₂ 배출량의 관계를 분석하고자 하므로 배기량이 0인 행을 삭제
- 다시 기초 통계량을 확인해 보면 데이터 개수는 1개 줄었고 평균, 표준편차 등의 통계량이 변해 있음

```
01 df = df[df['배기량']!=0]
02 print(len(df))
03 display(df.describe())
```

3497

	도심주행연비	고속도로연비	CO2배출량(g_km)	1회충전주행거리(km)	배기량	출시연도
count	3496.000000	3496.000000	3186.000000	402.000000	3186.000000	3497.000000
mean	9.622225	11.755978	168.908035	265.850498	2558.677966	2020.777238
std	4.492744	4.504737	48.694492	162.256235	964.282913	2.182431
min	1.600000	0.000000	14.000000	11.000000	796.000000	2008.000000
25%	7.300000	9.800000	137.000000	69.500000	1995.000000	2019.000000
50%	9.200000	11.800000	167.000000	303.000000	2199.000000	2021.000000
75%	11.500000	14.100000	199.000000	408.500000	2997.000000	2023.000000
max	99.500000	92.600000	354.000000	555.000000	6749.000000	2024.000000

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 시각화하기

- 휘발유 차량과 경유 차량을 따로 분석하기 위해 휘발유 차량 데이터는 df1에 저장하고 경유 차량 데이터는 df2에 저장

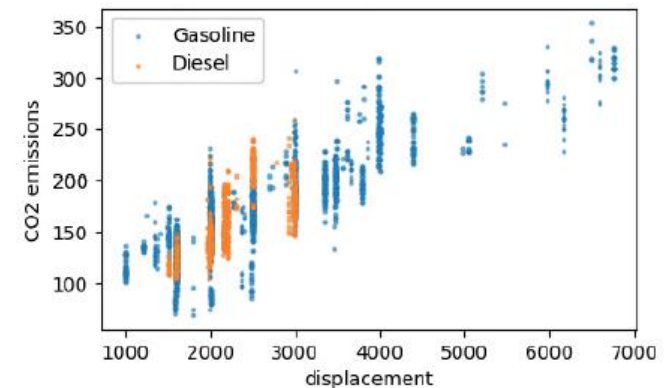
```
01 df1 = df[df['연료']=='휘발유']  
02 df2 = df[df['연료']=='경유']
```

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 시각화하기

- 휘발유 차량의 배기량-CO₂ 배출량 산점도와 경유 차량의 배기량-CO₂ 배출량 산점도를 하나의 그래프로 나타냄
- 연료 종류와 관계없이 배기량이 커짐에 따라 CO₂ 배출량이 커지는 것으로 나타남

```
01 plt.figure(figsize=(5,3))
02 plt.scatter(df1['배기량'], df1['CO2배출량(g_km)'], s=3, alpha=0.5, label='Gasoline')
03 plt.scatter(df2['배기량'], df2['CO2배출량(g_km)'], s=3, alpha=0.5, label='Diesel')
04 plt.legend()
05 plt.xlabel('displacement')
06 plt.ylabel('CO2 emissions')
07 plt.show()
```



5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 상관관계 분석하기

- 배기량과 CO₂ 배출량의 상관계수를 구하는 코드는 다음과 같음
- r을 출력해서 확인해 보면 두 변수에 양의 상관관계가 있고, 특히 휘발유 차량이 경유 차량보다 배기량과 CO₂ 배출량의 상관관계가 높다는 것을 알 수 있음

```
01 cor = df1['배기량'].corr(df1['CO2배출량(g_km)'])
02 print(cor)
03
04 cor = df2['배기량'].corr(df2['CO2배출량(g_km)'])
05 print(cor)
```

```
0.8563211276203075
0.6562481550600938
```

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 상관관계 분석하기

■ 전체 프로그램

```
01 import numpy as np
02 import pandas as pd
03 import matplotlib.pyplot as plt
04
05 df = pd.read_csv('car20240731.csv')
06 display(df.head())
07
```

5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 상관관계 분석하기

■ 전체 프로그램

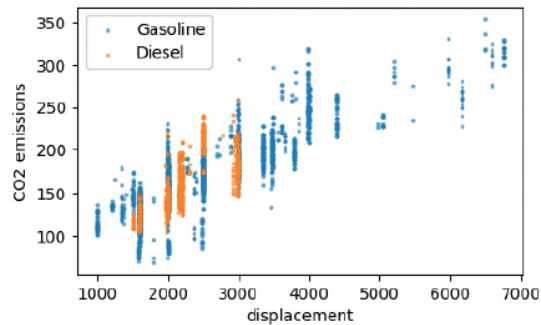
```
08 df = df[df['배기량']!=0]
09
10 df1 = df[df['연료']=='휘발유']
11 df2 = df[df['연료']=='경유']
12
13 plt.figure(figsize=(5,3))
14 plt.scatter(df1['배기량'], df1['CO2배출량(g_km)'], s=3, alpha=0.5, label='Gasoline')
15 plt.scatter(df2['배기량'], df2['CO2배출량(g_km)'], s=3, alpha=0.5, label='Diesel')
16 plt.legend()
17 plt.xlabel('displacement')
18 plt.ylabel('CO2 emissions')
19 plt.show()
20
21 cor = df1['배기량'].corr(df1['CO2배출량(g_km)'])
22 print(cor)
23
24 cor = df2['배기량'].corr(df2['CO2배출량(g_km)'])
25 print(cor)
```


5. [플러스 예제] 자동차 배기량과 CO₂ 배출량의 상관관계

■ 상관관계 분석하기

■ 전체 프로그램

	업체명	모델명	연료	변속형식	차량형식	자동차 종류	도심주행연비	고속도로연비	CO2배출량(g/km)	1회충전주행거리(km)	배기량	등급	승시연도
0	볼보	볼보 V60CCB5 AWD	휘발유	자동	하이브리드	승용차	8.7	11.9	175.0	NaN	1969.0	4등급	2024
1	볼보	볼보 XC60T8 AWD	전기+휘발유	자동	PHEV	승용차	16.6	17.1	33.0	61.0	1969.0	PHEV	2024
2	벤츠	Mercedes-Benz GLC 220 d 4MATIC	경유	자동	하이브리드	승용차	13.8	15.5	130.0	NaN	1993.0	2등급	2024
3	벤츠	Mercedes-Benz E 450 4MATIC	휘발유	자동	하이브리드	승용차	9.0	13.1	162.0	NaN	2998.0	4등급	2024
4	랜드로버	더 뉴 레인지로버 P550e SWB	전기+휘발유	자동	PHEV	승용차	11.5	13.1	35.0	80.0	2996.0	PHEV	2024



0.8563211276203075

0.6562481550600938